RAG AI Discord Bot

By: Jacob Hernandez

Abstract

Large Language Models (LLMs) often suffer from hallucinations and lack access to domain-specific knowledge. This project presents a Retrieval-Augmented Generation (RAG) AI Discord bot that is designed to improve response accuracy by integrating external knowledge sources into the generation process. The system retrieves relevant information from a curated knowledge base using semantic embeddings and vector search, and then generates context-aware responses via OpenAI API. The bot is deployed within Discord, enabling real-time interaction. Results demonstrate that the RAG-based approach produces more contextually grounded and reliable responses compared to standalone LLM outputs.

Introduction

Recent advances in artificial intelligence have enabled systems to produce highly coherent and human-like text. However, these models can generate problematic hallucinations where a LLM gives a confident but falsely correct response to someone. This limitation is especially an issue in environments where accurate, domain-specific knowledge is required.

Online communities, such as Discord servers, frequently encounter repeated questions related to documentation, technical topics, or group-specific knowledge. Traditional bots rely on static responses or keyword matching, which limits their flexibility and usefulness.

This project addresses these challenges by developing a RAG-based Discord bot that combines retrieval mechanisms with generative AI. By grounding responses in retrieved data, the system improves accuracy, contextual awareness, and reliability. The goal is to demonstrate a practical application of modern AI techniques in a real-world interactive setting.

Related Work

Retrieval-Augmented Generation (RAG) is a thoughtful approach to improving the factual accuracy of language models by incorporating external knowledge sources. LLMs rely on pre-trained parameters where domain-specific information is often isolated. RAG helps bridge the gap in connecting the knowledge to consider for a specific task.

Vector databases such as FAISS enable efficient similarity search over embedded text, making them a common component in RAG pipelines. Embedding models, such as those provided by SentenceTransformers, allow text to be represented in a semantic vector space, enabling meaning-based retrieval rather than keyword matching.

This project builds upon these ideas by integrating semantic retrieval, vector search, and generative modeling into a Discord bot. Unlike many research-focused implementations, this system emphasizes usability and real-time interaction within a messaging platform.

Data

The knowledge base for this project consists of two primary sources:

1. **Static Knowledge Files**
    - Stored in the `/data/knowledge_base/` directory.
    - Includes manually curated text files (e.g., project descriptions, technical notes).
    - Provides foundational knowledge for the system.
2. **Dynamic Discord Message Memory**
    - Messages from the Discord server are stored in a text file(`discord_memory.txt`).
    - Enables the bot to learn from previous interactions within the server.

Preprocessing

- Text data is split into smaller chunks for efficient retrieval.
- Each chunk is embedded using a sentence-transformer model.
- Embeddings are stored in a FAISS index for fast similarity search.

This approach ensures that both static and dynamic data can be queried efficiently.

Methods

System Architecture

The system follows a standard RAG pipeline:

User Query → Embedding → Vector Search (FAISS) → Retrieved Context → LLM Generation → Discord Response

---

4.1 System Overview

The system is designed as a Retrieval-Augmented Generation (RAG) pipeline integrated within a Discord bot. The architecture consists of three primary components: a user interface layer(main.py), a retrieval module(retriever.py), and a response generation module(generator.py). User queries are submitted through Discord, processed by the bot, and then passed through the retrieval/generation pipeline to produce context-aware responses.

When a user submits a query using the `!ask` command, the system first retrieves relevant information from a local knowledge base. This information is then combined with the user's query and recent conversation history to generate a final response using an AI language model. The modular design allows each component to operate independently, improving maintainability and scalability.

4.2 Knowledge Base and Preprocessing

The knowledge base consists of text documents stored locally in a structured directory. These documents are manually curated and formatted to contain relevant domain-specific information. To improve retrieval performance, each document is divided into smaller chunks based on paragraph separation. This chunking strategy ensures that the system retrieves concise and contextually relevant pieces of information rather than entire documents.

Each text chunk is cleaned by removing excess whitespace and stored in a list for further processing. This preprocessing step is critical for ensuring that the retrieval system operates efficiently and returns meaningful results.

Repeated facts improves the bot's confidence and accuracy when responding to a query. During development, the bot sometimes returns the statement of not having found the information a user was looking for. Repeated facts improve the fact's discoverability.

4.3 Embedding Model

To enable semantic search, the system converts text into numerical vector representations using a pre-trained model from SentenceTransformers. Specifically, the model “all-MiniLM-L6-v2” is used due to its balance between performance and computational efficiency.

This model maps both knowledge base text and user queries into a shared vector space, where semantically similar texts are located closer together. By using embeddings instead of keyword matching, the system can understand the meaning of queries even when different wording is used.

4.4 Similarity Search with FAISS

The system employs FAISS to perform efficient similarity search over the embedded knowledge base. All document embeddings are stored in a FAISS index using an L2 (Euclidean distance) metric.

When a user submits a query, it is converted into an embedding and compared against the indexed document vectors. The system retrieves the top *k* most similar text chunks based on distance. These retrieved documents represent the most relevant context for answering the user’s question.

The use of FAISS significantly improves retrieval speed and scalability, allowing the system to handle larger knowledge bases while maintaining fast response times.

4.5 Response Generation

After retrieving relevant context, the system generates a response using an AI language model accessed via the OpenAI API. The prompt provided to the model includes three key components: the retrieved context, the user’s current question, and recent conversation history.

The model used is “gpt-4o-mini,” which offers a balance between response quality and computational efficiency. A low temperature setting (0.3) is used to reduce randomness and improve the consistency of generated answers.

By combining retrieved knowledge with generative capabilities, the system ensures that responses are both contextually grounded and conversationally coherent.

4.6 Conversation Memory

The system maintains a limited chat history of recent user queries and generated responses. This history is included in the prompt sent to the language model, enabling it to generate more context-aware answers.

To prevent excessive memory usage and maintain performance, the history is capped at a fixed number of interactions. Additionally, a command is provided to clear the conversation history when needed.

4.7 System Integration

The entire system is implemented in Python and leverages the Discord API for real-time communication. Environment variables are managed securely using a `.env` file, ensuring that sensitive information such as API keys is not exposed in the source code.

The modular architecture with the separation of retrieval, generation, and interface logic allows for easy future enhancements, such as replacing the retrieval method or upgrading the language model.

Experiments & Results

Qualitative Evaluation

The system was evaluated through interactive testing within Discord. Example queries include:

- “What are we building?”
- “What technologies does this project use?”
- “Why is RAG useful?”

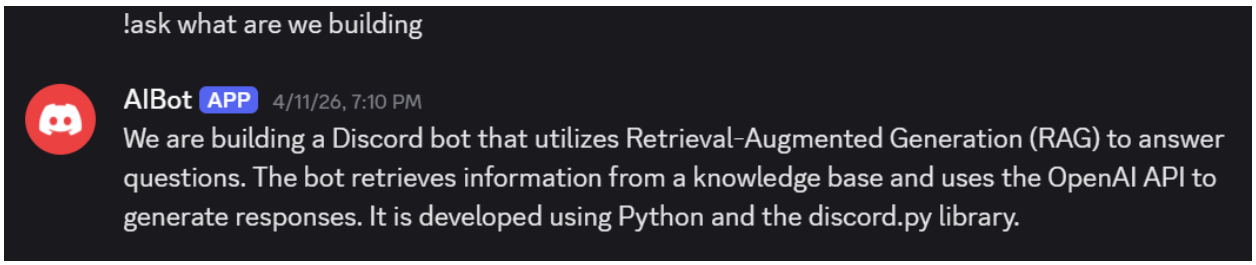
Results show that:

- The bot successfully retrieves relevant information from the knowledge base
- Responses are grounded in provided data rather than generic AI outputs
- Follow-up questions benefit from conversation memory

Example

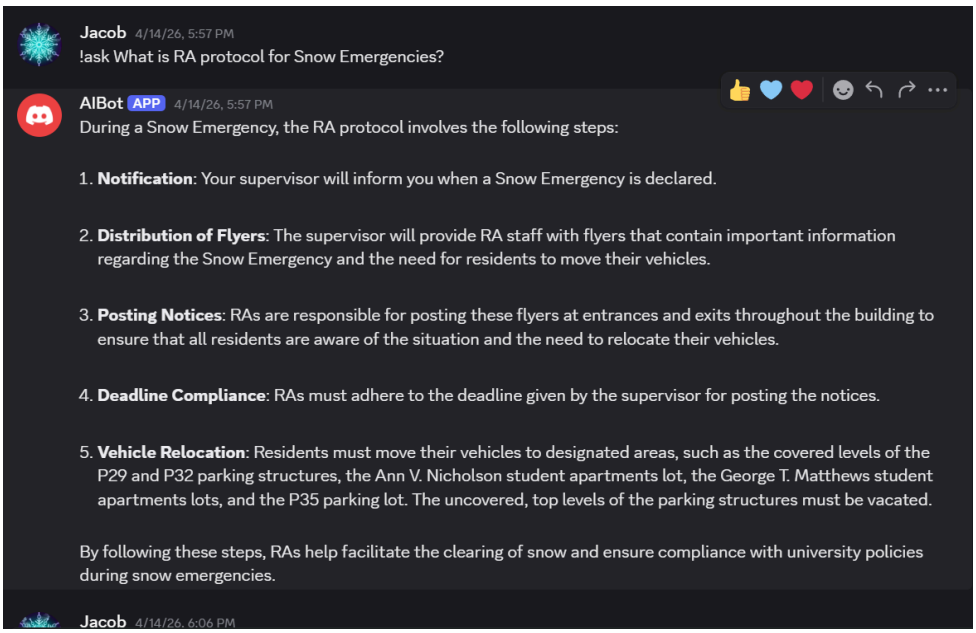
Input:
`!ask what are we building.`

Output:
A response describing the RAG AI Discord bot architecture.



Input:
`!What is RA protocol during Snow Emergencies?`

Output:
A response describing the protocol



The bot referenced this info from the txt.file:

```
SNOW EMERGENCY:
A snow emergency is usually declared by the University when we are expecting to experience
severe weather. During snow emergencies, all residential vehicles must be moved to the covered
levels of the P29 and P32 parking structures (note: the uncovered, top levels of the structures are
exempt from this policy and must be vacated during a snow emergency.), the Ann V. Nicholson
student apartments lot, the George T. Matthews student apartments lots, and P35 parking lot.
This is necessary for snow plows to come in and clear out our standard lots.

Vehicles left in any other main campus lots during a snow emergency will be impounded at the
owner's expense. OU will not assess a fine for Policy 483 violations, but retrieval of impounded
vehicles will require payment of a towing fee and may also involve storage fees, charged per day.
Housing's role in this process is to ensure that flyers and notices to residents about moving their
cars are being properly posted/sent out. Your supervisor will reach out and inform you when
there is a Snow Emergency. They will provide RA staff with flyers, and a deadline to have them
posted on entrances and exits throughout the building.
```

Observations

- The RAG system uses context from messages in the discord server and info stored in the info.txt file to frame its response.
- Retrieval quality depends on the relevance and structure of stored data.
- Semantic embeddings allow flexible, meaning-based queries.

Limitations

- Retrieval quality may degrade with poorly formatted data.
- Currently is only able to reference data from the txt. file that is accessible before the bot is actively connected to the discord server.

Conclusion

This project developed a context-aware Discord bot capable of answering user questions using a custom knowledge base. By integrating AI-generated responses with structured data, the system provides a more dynamic and interactive alternative to traditional static information sources.

The bot demonstrated strong performance when handling well-defined queries and highlighted the importance of high-quality data in AI systems. However, several limitations were identified, including reliance on manual data formatting and reduced accuracy for ambiguous inputs.

Future improvements could include implementing more advanced data retrieval techniques, expanding the knowledge base, and enhancing context management. Additionally, incorporating user feedback mechanisms could further improve system performance over time.

Future Work

Improvements I plan to include over summer include:

- Integration of more data.
- Implementation of more advanced data retrieval techniques.
- Incorporation of AIBot personalization.
- Integration of LLM enhancement into other discord bot capabilities.
- Deployment into multiple discord servers.

Citations & Acknowledgements

- Discord API documentation Discord Developer Platform - Documentation - Discord
- OpenAI API **https://platform.openai.com/**

- ChatGPT for development guidance and assistance in debugging and fixing the code files.
- Course materials and project guidelines